Computer-System Organization and Architecture

Week 2 — Interrupts, Storage Structure, I/O Structure, and DMA

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-502: Operating Systems

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

October 17, 2026

Computer Net Click

Welcome: Five Events Before One Character Appears

The big question

You press `Enter` to run `cat bigfile.txt`. Before a single byte reaches the screen, the keyboard controller, the CPU, the memory, the disk controller, and the bus must all cooperate. Which of them can signal the CPU — and how?

- The OS does not run "on its own". It is *event-driven*: it reacts to hardware events it did not initiate.
- Last week: what the OS *is*. This week: the machine it stands on, and the two mechanisms — **interrupts** and **DMA** — that make it possible to stand there.

Running thread for today

We follow the command `cat bigfile.txt` from the Enter key to the disk platter and back: an interrupt starts it, a storage hierarchy feeds it, and DMA moves the bytes.

Roadmap: Five Questions About the Machine

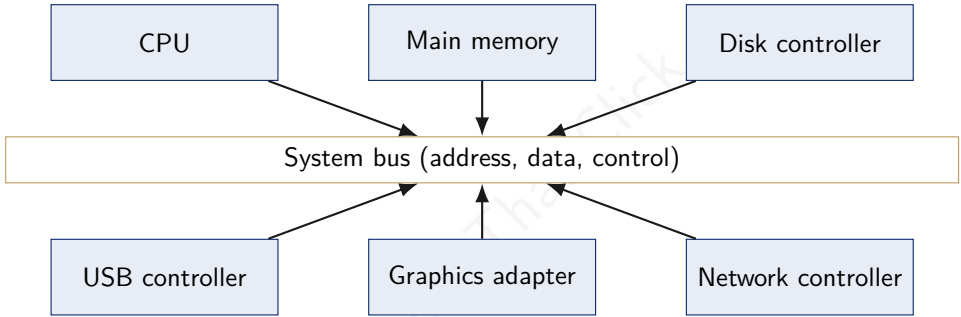
1. **What is the machine made of?** — CPU, memory, controllers, bus.
2. **How does hardware get the CPU's attention?** — interrupts, traps, and the interrupt vector.
3. **Where do data live?** — main memory, disks, SSDs, and the memory hierarchy.
4. **How do bytes move?** — polling, interrupt-driven I/O, and DMA.
5. **How does the machine scale?** — multiprocessors, multicore, and clusters.

Every answer is an OS-visible fact: each mechanism in this lecture reappears later as a scheduling, memory, or I/O policy.

Concepts That Click

A computer is not a CPU that does everything. It is a set of independent devices that run concurrently and communicate over a shared bus.

Anatomy of a Computer System



- Each device has a **controller** — hardware that speaks the bus protocol and operates the device.
- The CPU and controllers run **concurrently**; the bus is the shared resource they contend for.

Bootstrap: How the Machine Starts

1. **Firmware** (BIOS/UEFI) runs from ROM/flash when power is applied. It tests the hardware (POST) and finds a boot device.
2. The **bootstrap program** loads the OS kernel from disk into memory and jumps to its entry point.
3. The kernel initializes its data structures, starts the first process (`init` / `systemd`), and from then on is **event-driven**.

Why the OS cannot assume it is running

Until the kernel is loaded, no OS exists. After it is loaded, the kernel only runs when a *trap*, an *interrupt*, or an explicit call gives it the CPU. The next section is about that handoff.

Bootstrap sequence: Silberschatz et al., Ch. 1 (pp. 6–8); Stallings, Ch. 1 [3].

The CPU's View: Fetch, Decode, Execute — and Wait

- The CPU repeats one cycle: **fetch** an instruction at the program counter, **decode** it, **execute** it, update the PC.
- Registers hold the working set: general-purpose registers, the **program counter (PC)**, the stack pointer, and the status word.
- I/O is orders of magnitude slower than the CPU. A 3 GHz core executes roughly 3 instructions per nanosecond; a disk access takes milliseconds.

The utilization problem

If the CPU waited for every device, it would idle almost all the time. The OS needs a way to *start* an I/O operation and be told later that it finished — without watching it.

Instruction cycle and registers: Stallings, Ch. 1 §1.2–1.3 (pp. 7–15) [3].

Controllers and Drivers: Hardware and Its Software Twin

Device controller (hardware)

A chip or board with registers the CPU can read and write: **status** (busy, ready, error), **control** (start, reset, mode), and **data-in**/**data-out** buffers. It operates the physical device.

Device driver (software)

Kernel code that knows the controller's register layout and protocol. The OS calls the driver; the driver programs the controller; the controller moves the data.

- One driver per controller family; the rest of the kernel sees a uniform interface (read, write, ioctl).
- The kernel keeps a **device-status table** listing each device, its type, address, and state.

Controllers, drivers, and the device-status table: Silberschatz et al., Ch. 1 (pp. 6–18).

Worked Thread: The Enter Key, Step by Step

1. You press `Enter`. The keyboard controller scans the key matrix and places a scan code in its data register.
2. The controller asserts an **interrupt request (IRQ)** line on the bus.
3. The CPU finishes the current instruction, saves its state, and jumps to the keyboard **interrupt service routine (ISR)**.
4. The ISR reads the scan code, maps it to a character, and queues it for the terminal process.
5. The scheduler eventually runs the shell, which reads the character and starts `cat bigfile.txt` — which now needs the disk.

The pattern to remember

Device → controller → IRQ → CPU saves state → ISR → kernel decides what happens next. Every device in the machine uses a variant of this path.

Socratic Check 1

Think, then answer

Why can the OS not simply poll every device in a loop, checking whether each one is ready? Name two distinct costs of that design.

- **CPU cost**: polling consumes cycles that could run user code — and most polls find nothing ready.
- **Latency cost**: a slow or sporadic device (a keyboard, a network packet) may wait an entire polling period before being noticed.
- Interrupts invert the arrangement: the device tells the CPU when it needs attention, instead of the CPU asking.

The CPU is fast and the devices are slow and independent. The only workable contract is: finish your instruction, then let the device interrupt you.

Concept by Click

Interrupts vs. Traps: Two Kinds of Transfer

Interrupt (asynchronous)

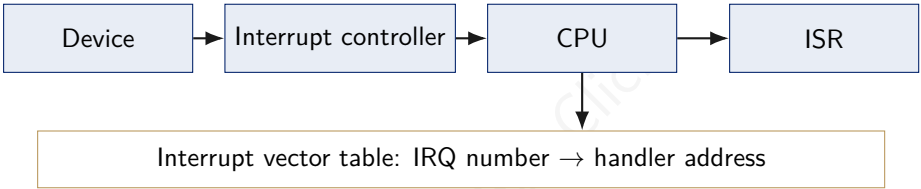
Raised by *hardware* at an unpredictable time: a key press, a packet arrival, a disk completing a transfer. It is unrelated to the instruction the CPU is currently executing.

Trap or exception (synchronous)

Raised by the *CPU* because of the instruction just executed: a system call, a divide by zero, or an illegal opcode. It is caused by the program itself.

- Both transfer control to a kernel handler and both save the CPU state.
- The distinction matters: an interrupt can arrive *between* any two instructions; a trap is tied to a specific instruction.

The Interrupt Path



1. Device asserts IRQ; the interrupt controller prioritizes and forwards it.
2. CPU finishes the current instruction (not the whole program), saves the PC and status word, and switches to kernel mode.
3. CPU indexes the **interrupt vector table** by IRQ number and jumps to the handler.
4. The ISR services the device; on return, the saved state is restored.

The Interrupt Vector Table, Masking, and Priority

- The **interrupt vector table** is an array of handler addresses, indexed by interrupt number. It lets the CPU dispatch without searching.
- **Maskable** interrupts can be temporarily disabled by the kernel (when updating shared structures). **Non-maskable** interrupts (power failure, fatal hardware error) cannot.
- **Priority levels** decide which interrupt wins when several arrive at once; a low-priority handler can itself be interrupted by a high-priority one.

The design tension

Handlers must be short, or they block other interrupts and add latency. So real kernels split work: a minimal **top half** acknowledges the device and a deferred **bottom half** does the rest (Linux softirqs, tasklets, work queues).

Linux top-half/bottom-half handling: Bovet & Cesati, Ch. 4 (pp. 131–188) [1].

Worked Example: What Does Interrupt Handling Cost?

Setup

Saving the context and entering the handler costs $2\ \mu$s; the handler itself runs for $8\ \mu$s. Each interrupt therefore costs $10\ \mu$s of CPU time.

- At **1,000 interrupts/s**: $1{,}000 \times 10\ \mu\text{s} = 10$ ms/s $= 1\%$ CPU overhead.
- At **10,000 interrupts/s**: $10{,}000 \times 10\ \mu\text{s} = 100$ ms/s $= 10\%$ CPU overhead.
- At **100,000 interrupts/s**: the machine spends 100% of its time handling interrupts and does no useful work.

The lesson

Interrupts are cheap per event but not free. When a device produces a *stream* of events, per-event interrupts stop scaling — which is exactly the problem DMA solves for block transfers.

Socratic Check 2

Think, then answer

An interrupt handler runs with interrupts temporarily disabled. The handler is long. What can go wrong, and what does this imply about handler design?

- Other devices cannot deliver their interrupts, so their data can be overwritten or lost — a **missed interrupt**.
- The system's **interrupt latency** grows; a real-time device may miss its deadline.
- Hence the top-half/bottom-half split: acknowledge quickly, defer the heavy work to a context where interrupts are enabled.

Interrupts tell the CPU that data has arrived. But where does the data live, and why can a program believe it has more memory than the machine owns?

Main Memory and Secondary Storage

Main memory (RAM)

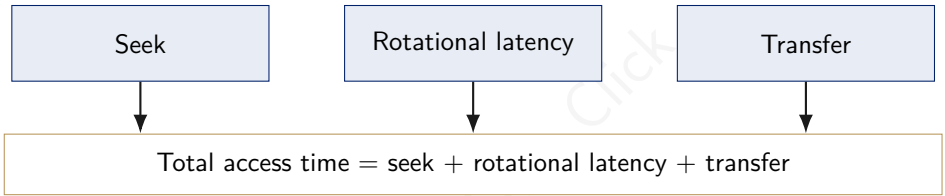
A large array of bytes, each with an address, implemented in DRAM. Fast (tens of nanoseconds) and directly addressable by the CPU; volatile — its contents vanish when power is lost.

Secondary storage

Hard disks (magnetic platters) and **solid-state drives** (NAND flash). Non-volatile and far larger than RAM, but orders of magnitude slower — microseconds for an SSD, milliseconds for a spinning disk.

- Programs live in memory only while they run; files persist on secondary storage.
- The OS moves data between the two in units of **blocks** (disk) or **cache lines** (memory).

Hard-Disk Geometry: Where Access Time Comes From



- **Platters** rotate; each surface has concentric **tracks** split into **sectors**; the same track across platters is a **cylinder**.
- **Seek**: move the arm to the track (mechanical, milliseconds).
- **Rotational latency**: wait for the sector to pass under the head (milliseconds).
- **Transfer**: stream the requested sectors (microseconds per block).

Worked Example: A 4 KB Disk Read at 7200 RPM

Parameters

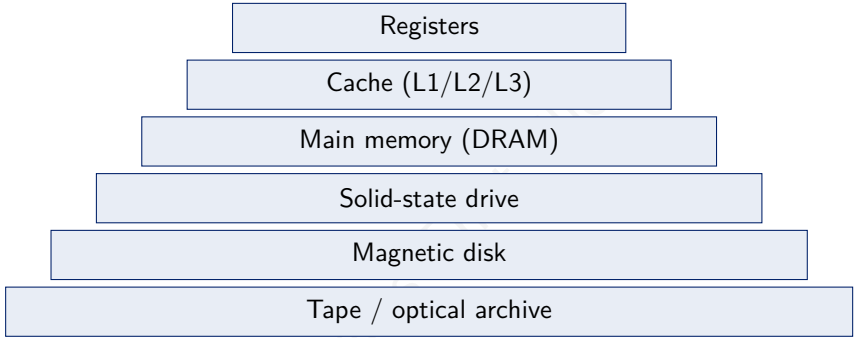
Seek time $= 8$ ms; rotation speed $= 7{,}200$ RPM; transfer rate $= 150$ MB/s; block size $= 4$ KB.

- One rotation: $60/7{,}200 = 8.33$ ms; average rotational latency $= 8.33/2 = 4.17$ ms.
- Transfer time: 4 KB$/150$ MB/s ≈ 27 μs.
- Total: $8 + 4.17 + 0.027 \approx 12.2$ ms, of which *99.8%* is mechanical waiting.

The consequence for the OS

Disk performance is dominated by **where the head is**, not by how fast bytes stream. This is why scheduling the *order* of requests (later, disk scheduling) matters so much.

The Memory Hierarchy



- Moving down: capacity grows, cost per byte falls, access time grows by orders of magnitude.
- Moving up: data is **cached** — a copy is kept in the faster level on the bet that it will be used again.

Locality, Caching, and Why the Hierarchy Works

The locality principle

Programs do not access memory uniformly. **Temporal locality**: an item used once is likely to be used again soon. **Spatial locality**: an item near one just used is likely to be used soon.

- A **cache** is a small, fast store holding copies of recently used data. A **hit** is served from the cache; a **miss** goes to the next level and installs a copy.
- The OS manages caches at every level: CPU caches (hardware), the **buffer cache** for disk blocks, and memory itself as a cache for disk.

Caching and the memory hierarchy: Stallings, Ch. 1 §1.5–1.6 (pp. 26–32) [3]; Silberschatz et al., Ch. 1 (pp. 9–14).

Socratic Check 3

Think, then answer

Main memory is volatile, yet a file you saved last week still exists after a power cut. Where is the file, and what role did RAM play in saving it?

- The file lives on **secondary storage** (disk or SSD), which is non-volatile.
- RAM held the program's buffer and the kernel's **cache** while the write was in progress; the OS copied the dirty blocks to storage.
- This is why "save" is an I/O operation, not a memory operation — and why a crash before the write-back can lose data.

Data must come from a device into memory. The question is who moves it: the CPU, one byte at a time, or a controller acting on its own.

Three Ways to Move Data

	Polling	**Interrupt-driven**	**DMA**
Data mover	CPU	CPU	Controller
CPU cost	Wasted while waiting	One handler per event	Setup + one completion
Best for	Fast, dedicated devices	Slow, sporadic devices	Block transfers
Weakness	Busy-wait; latency	Overhead per event	Setup cost; bus contention

- The three techniques are a progression: the CPU does *more* waiting, then *less* work, then *no* per-byte work.
- `cat bigfile.txt` uses a **block device**: DMA is the natural choice; a keyboard uses **interrupt-driven** I/O.

Concepts First Click

Polling: Simple but Wasteful

Programmed I/O (polling)

The CPU repeatedly reads the controller's **status register** until the device is ready, then moves the data word by word.

- Simple: no interrupt machinery, no concurrency.
- Wasteful: every poll is a bus transaction and a wasted instruction slot; a slow device wastes almost all of them.
- Still right for very fast devices (a GPU register) or when the wait is predictably short — polling a device that is always ready beats paying interrupt overhead.

Programmed I/O: Silberschatz et al., Ch. 1 §1.2.3 (pp. 12–18); Stallings, Ch. 1 §1.7 (p. 33) [3].

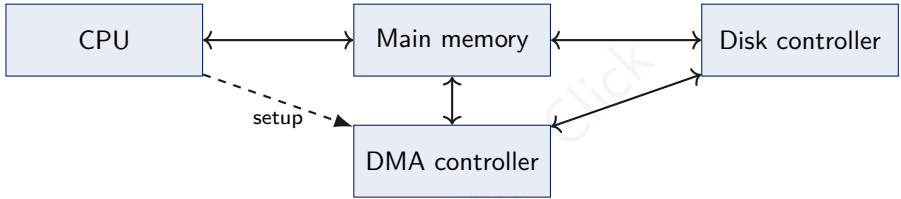
Interrupt-Driven I/O: The CPU Is Notified

1. The driver starts the device by writing its control register.
2. The CPU returns to other work; the scheduler runs another process.
3. When the device finishes, it raises an interrupt.
4. The ISR reads the data (or a status), acknowledges the device, and wakes the waiting process.

The cost model

CPU cost scales with the *number of events*, not the number of bytes. Small blocks therefore mean many interrupts; large blocks mean few. This is the lever DMA pulls.

Direct Memory Access (DMA)



1. The CPU programs the DMA controller with the memory address, the byte count, and the device.
2. The controller transfers the whole block directly to or from memory, stealing bus cycles but no CPU instructions.
3. The controller raises *one* interrupt when the block is complete.

DMA: Silberschatz et al., Ch. 1 §1.2.3; Stallings, Ch. 1 §1.7 (p. 33) [3].

Worked Example: Moving 1 MB Three Ways

Parameters

Disk transfer rate $= 200$ MB/s; interrupt cost $= 10$ μs; DMA setup + completion interrupt ≈ 20 μs; transfer time for 1 MB $= 5$ ms.

- **Interrupt-driven, 4 KB blocks**: 256 interrupts $\times 10$ μs $= 2.56$ ms — about 51% of the transfer.
- **Interrupt-driven, 64 KB blocks**: 16 interrupts $\times 10$ μs $= 160$ μs — about 3.2%.
- **DMA**: one setup plus one completion interrupt ≈ 20 μs — about 0.4%, while the CPU runs other work.

The trade-off, stated precisely

Larger blocks amortize interrupt overhead but increase latency and buffer memory. DMA removes per-byte CPU work but adds setup cost — so it wins on large transfers, not on a single byte.

Device Types and the Device-Status Table

- **Block devices**: addressable in fixed-size blocks, seekable — disks, SSDs, USB drives. They support file systems and DMA.
- **Character devices**: a stream of bytes, not seekable — keyboards, serial ports, mice. They are naturally interrupt-driven.
- The kernel maintains a **device-status table**: for each device, its type, address, and current state (idle, busy, error).
- When an interrupt arrives, the kernel looks up the device in the table, updates its state, and wakes any process waiting on it.

Device types and the device-status table: Silberschatz et al., Ch. 1 (pp. 12–18).

Concepts That Click

Socratic Check 4

Think, then answer

DMA removes the CPU from the data path. Does that mean a DMA transfer uses no CPU time at all? Name the hidden costs.

- **Setup**: the CPU programs address, count, and direction for every transfer.
- **Bus contention**: the DMA controller steals memory cycles, slowing the CPU's own memory accesses (cycle stealing).
- **Cache coherence**: the CPU may hold stale copies of memory the DMA wrote; the OS and hardware must invalidate or flush.
- Conclusion: DMA is a *trade* of per-byte CPU work for per-block setup and bus pressure — not a free lunch.

One CPU, one memory, one disk is the simple case. Real machines replicate processors and whole systems — and the OS must keep them coherent.

From One Processor to Many

Single-processor systems

One general-purpose CPU. Most personal computers and phones have several cores per chip, but the OS still sees a set of processors to schedule.

- **Multiprocessor** systems: two or more processors share the bus, clock, and memory.
  - **SMP (symmetric multiprocessing)**: every processor is equal and runs the same OS; the scheduler can place any task on any CPU.
  - **Asymmetric**: one master processor assigns work to slaves — simpler, but the master is a bottleneck.
- **Multicore**: several independent cores on one chip, each with its own registers and usually its own L1/L2 cache, sharing a last-level cache and memory.

Clusters, NUMA, and Why the OS Cares

- **Cluster**: two or more complete systems connected by a network, sharing storage, presenting as one service. Used for **high availability** (failover) and **high-performance computing** (parallel work).
- **NUMA (non-uniform memory access)**: in large multiprocessors, memory is attached to nodes; a core accesses local memory faster than a remote node's memory. The scheduler and allocator should keep a task near its data.
- **Why it matters**: the OS must decide *where* to run a task and *where* to place its memory — decisions that do not exist on a single-processor machine.

The recurring theme

Every layer of hardware organization — cache, memory, bus, processor — becomes a policy question for the OS: which copy, which CPU, which node, which order.

Comparison: SMP, Multicore, and Clusters

	SMP	**Multicore**	**Cluster**
What is shared?	Bus, clock, memory	Last-level cache, memory	Network, storage
Failure unit	One processor	One core	One whole machine
Main benefit	Throughput, scaling	Throughput, lower latency	Availability, scale-out
OS challenge	Scheduling, locking	Cache affinity, locking	Distributed coordination

- All three give **parallelism**; only clusters survive the failure of an entire machine.
- All three make **concurrency** real for the OS — the subject of Unit II.

Concepts that Click

Socratic Check 5

Think, then answer

A machine has four cores but a single memory bus. Two programs each read a 1 GB array. What limits the speedup, and what would you change first?

- The **memory bus** is the shared bottleneck: more cores do not add memory bandwidth. The workload is **memory-bound**, not CPU-bound.
- First fixes: improve **locality** (block the loops so each cache line is reused), then add cache or bandwidth — not more cores.
- This is why the OS and compiler care about caches and NUMA placement, not just the number of processors.

Takeaways

- A computer system is a **CPU, memory, controllers, and a bus** — independent devices that run concurrently.
- Hardware gets the CPU's attention with an **interrupt** (asynchronous); the CPU gets the kernel's attention with a **trap** (synchronous).
- The **interrupt vector** dispatches to a handler; handlers must be short, hence top-half/bottom-half designs.
- Storage is a **hierarchy**: registers, cache, RAM, SSD, disk, tape — faster and costlier per byte as you move up, kept coherent by **caching** and locality.
- I/O moves data by **polling**, **interrupts**, or **DMA**; the choice trades CPU work against latency, overhead, and bus pressure.
- **SMP, multicore, and clusters** replicate processors or machines; the OS must schedule and place work across them.

The Thread in One Picture

1. **Keyboard interrupt**: the controller signals, the ISR queues a character, the shell runs.
2. **Storage hierarchy**: `cat` asks for file blocks; the buffer cache checks RAM before touching the SSD or disk.
3. **DMA**: the disk controller moves the block into memory; the CPU is interrupted once when it finishes.
4. **Architecture**: the scheduler may run `cat` on another core while the DMA transfer proceeds — and must keep the caches coherent.

One sentence

The OS is an event-driven manager of a hierarchy of asynchronous devices — interrupts tell it *when*, storage tells it *where*, and DMA tells it *how* to move data without doing the moving itself.

Next: OS Operations and System Protection

- Today: the hardware the OS manages and the signals it responds to.
- Next lecture: how the OS *stays in control* — dual-mode operation, the timer, privileged instructions, and the protection boundary that makes every mechanism in this lecture safe to share.

Reading

Silberschatz et al., *Operating System Concepts*, Ch. 1 §1.5 (pp. 20–23) and Ch. 1 §1.2 (pp. 6–18) [2]; Stallings, *Operating Systems: Internals and Design Principles*, Ch. 1 §1.4–1.7 (pp. 15–38) [3]. Optional concrete depth: Bovet & Cesati, *Understanding the Linux Kernel*, Ch. 4 (pp. 131–188) [1].

Further Reading (Page-Cited)

- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 8e — Ch. 1 §1.2 (computer-system organization, interrupts, storage, I/O, DMA, pp. 6–18), §1.3 (architecture, pp. 15–20), §1.7–1.8 (memory and storage management, pp. 24–26).
- W. Stallings, *Operating Systems: Internals and Design Principles*, 6e — Ch. 1 §1.4 (interrupts, p. 15), §1.5 (memory hierarchy, p. 26), §1.6 (cache, p. 29), §1.7 (I/O techniques and DMA, p. 33).
- D. P. Bovet & M. Cesati, *Understanding the Linux Kernel*, 3e — Ch. 4 (interrupts and exceptions, pp. 131–188), optional.

Page-verified via each book's `index.md` under `master_supporting_docs/CS502/supporting_books/`.

References I


Daniel P. Bovet and Marco Cesati.
Understanding the Linux Kernel.
O'Reilly Media, 3rd edition, 2006.


Abraham Silberschatz, Peter B. Galvin, and Greg Gagne.
Operating System Concepts.
John Wiley and Sons, 2018.


William Stallings.
Operating Systems: Internals and Design Principles.
Prentice Hall, 2001.

Concepts That Click